

# Sew Protocol Whitepaper

---

**Version:** 1.0

**Last Updated:** 2026-01-28 (Added Fee Snapshot Immutability)

**Network:** Base (Ethereum L2)

**Status:** Pre-Mainnet

---

## Vision & Why

We exist to make Ethereum usable for everyday payments. The primary blocker is not throughput or UX — it is risk: users cannot safely transact for physical goods without protection against fraud, non-delivery, or mistakes.

Sew introduces Ethereum-native payment protection. It provides onchain escrow, resolution, and accountability without custodians, surveillance, or platform control. Users retain custody, contracts enforce outcomes, and disputes are handled by decentralized resolver networks rather than centralized intermediaries.

The protocol adds new primitives for conditional settlement, dispute escalation, and incentive-aligned resolution. These integrate with account abstraction but also work with standard EOAs, making Sew deployable across today's Ethereum ecosystem.

Sew is open-source, non-custodial, and governance-controlled — designed to protect people, not platforms.

---

## Executive Summary

Sew Protocol is a decentralized escrow and dispute-resolution layer on Base for real-world commerce.

It enables peer-to-peer Ethereum payments for physical goods with:

Trustless settlement: funds lock and release by contract rules  
Trust-minimized dispute resolution: escalation + incentives + accountable resolver tiers  
Optional yield: bounded by caps and pause controls  
Time-delayed governance: upgrades apply to new escrows only (snapshot immutability)

Sew removes the need for marketplaces, chargebacks, or custodians by embedding buyer and seller protection directly into smart contracts. Funds are locked, conditions are enforced, and disputes are resolved by economically incentivized, accountable resolvers.

The result is a payment primitive that makes Ethereum viable for everyday transactions without sacrificing decentralization, self-custody, or transparency.

**Primary Use Case:** Safe everyday purchases of physical goods. Sew allows consumers to pay with Ethereum while being protected against fraud, non-delivery, and errors — enabling trustless commerce without intermediaries.

### Key Innovations:

- **Modular Architecture:** Pluggable resolution, yield, and distribution modules
- **Snapshot Immutability:** Escrow rules locked at creation, immune to governance changes
- **Multi-Token Support:** Escrow any ERC20 token with a single interface
- **Optional Yield Generation:** Earn yield on escrowed funds via Aave integration
- **Decentralized Dispute Resolution:** Multi-level escalation system with fair resolver selection
- **Governance-Controlled Evolution:** Time-delayed, transparent protocol upgrades

# Release & Activation Matrix (Authoritative)

This document distinguishes four separate “truths” and makes status explicit:

- **Code status:** implemented in this repository
- **Testing status:**
  - (a) local tests (unit / fuzz / invariants)
  - (b) testnet validation (deployment + scenario runs)
  - (c) simulation / chaos testing with published results
  - (d) external audit(s)
- **Deployment status:** whether contracts are deployed to a network
- **Activation scope:** whether a component is active by default and whether it applies to new escrows only (snapshot immutability)

Strict status terms (used everywhere)

- **Deployed** = contract exists on a network
- **Activated** = selected/used for **new escrows by default** (or selected module), via governance where applicable
- **Validated** = testnet and/or simulation validation performed with documented results (separate from local unit/fuzz/invariant tests)

Initial mainnet release vs DR phases (canonical)

**Initial mainnet release (IEO launch configuration):** Core escrow contracts + governance + `DefaultResolutionModule` (single resolver).

**DR v1/v2/v3 modules:** Implemented in code and passing local test suites, but **not yet validated on testnet or through simulation testing** at the time of this document. They are **not active** in the initial mainnet release.

**Activation method:** DR modules are activated only via a **governance-activated module swap** and affect **new escrows only** due to snapshot immutability.

Matrix

| Component                                                                                         | Code Status | Testing Status                                 | Deployment Status             | Activation Scope                        |
|---------------------------------------------------------------------------------------------------|-------------|------------------------------------------------|-------------------------------|-----------------------------------------|
| Core escrow ( <code>BaseEscrow</code> , <code>EscrowVault</code> , <code>EscrowableERC20</code> ) | Implemented | Local unit/integration/fuzz/invariants passing | Planned for mainnet           | Applies to all escrows                  |
| Governance (Governor, Timelock, Guardian)                                                         | Implemented | Local unit/integration passing                 | Planned for mainnet           | Applies to all governance actions       |
| <code>DefaultResolutionModule</code> (single resolver)                                            | Implemented | Local tests passing                            | Initial mainnet launch module | Applies to new escrows created under it |

| Component                             | Code Status | Testing Status                         | Deployment Status               | Activation Scope                                           |
|---------------------------------------|-------------|----------------------------------------|---------------------------------|------------------------------------------------------------|
| DR v1 module (decentralize decisions) | Implemented | Local unit/fuzz/invariants passing     | Not yet testnet/sim validated   | Governance-activated module swap; affects new escrows only |
| DR v2 module (appeal bonds)           | Implemented | Local unit/fuzz/invariants passing     | Not yet testnet/sim validated   | Governance-activated module swap; affects new escrows only |
| DR v3 module (staking/slashing)       | Implemented | Local unit/fuzz/invariants passing     | Not yet testnet/sim validated   | Governance-activated module swap; affects new escrows only |
| Aave yield module                     | Implemented | Local unit/integration/fuzz/invariants | Optional at launch (if enabled) | Per-escrow enablement + caps                               |

## 1. Introduction

### 1.1 The Trust Problem in Blockchain Transactions

Blockchain technology has revolutionized digital transactions by enabling trustless, peer-to-peer value transfer. However, this trustlessness creates a fundamental challenge: **onchain transactions are irreversible**. While this immutability provides security and finality, it creates significant friction for everyday purchases of physical goods where:

- **Buyers** need protection against non-delivery, damaged goods, or misrepresented items
- **Sellers** need assurance of payment before shipping physical products
- **Both parties** need a mechanism to resolve disputes fairly without relying on centralized intermediaries

Traditional solutions rely on centralized intermediaries (e.g., payment processors, escrow services, marketplaces), which reintroduce trust assumptions, monitoring, unclear terms, and lock-in - defeating the purpose of decentralized systems.

### 1.2 Current State of Escrow on Ethereum

Existing escrow solutions on Ethereum suffer from several limitations:

1. **Fragmentation:** Each marketplace or protocol implements custom escrow logic, creating silos
2. **Lack of Standardization:** No common interface for escrow operations
3. **Limited Dispute Resolution:** Most solutions rely on manual multisig coordination
4. **No Composability:** Difficult to integrate escrow into existing dApps
5. **Poor User Experience:** Complex, inconsistent interfaces across platforms

### 1.3 Our Solution

Sew Protocol provides a **standardized, modular, and governance-controlled** escrow system that:

- Enables secure, reversible payments for everyday purchases of physical goods
- Provides built-in dispute resolution with multiple escalation levels
- Supports automated time-based settlements
- Generates optional yield on escrowed funds
- Maintains composability with account abstraction and existing DeFi protocols
- Works with both account abstraction wallets and legacy EOA wallets
- Evolves transparently through onchain governance
- No custodians, no user surveillance, and no platform lock-in — rules are enforced onchain, any module swapping is transparent and time-delayed

---

## 2. Protocol Overview

### 2.1 Non-Goals

To clarify scope and reduce liability-style misreadings, Sew Protocol is explicitly **not**:

- **An oracle of truth:** Does not verify off-chain facts or authenticate goods/services
- **A shipping provider:** Does not track packages or coordinate delivery
- **An identity/KYC system:** Does not verify identities or perform Know Your Customer checks
- **Consumer law enforcement:** Does not enforce legal regulations or replace legal recourse

Sew is a **payment and dispute resolution primitive** that provides escrow functionality and cryptoeconomic dispute resolution. Off-chain verification, shipping, identity, and legal compliance remain the responsibility of users or integrated services.

### 2.2 Core Principles

The protocol is built on four fundamental principles:

1. **Trustlessness:** No single party controls funds or can unilaterally modify escrow rules
2. **Immutability:** Escrow rules, modules, and fees are snapshotted at creation and cannot be changed during the escrow's lifetime
3. **Modularity:** Pluggable modules for resolution, yield, and distribution
4. **Transparency:** All operations are onchain and publicly verifiable

### 2.2 Architecture Overview

Sew Protocol is built on a modular architecture that enables safe, trustless transactions for everyday purchases:

#### Core Escrow Contracts (Immutable)

- **BaseEscrow:** Abstract base contract with shared escrow logic, state machine, and module integration

- **EscrowVault:** Multi-token escrow vault supporting any ERC20 token - ideal for marketplaces and multi-token use cases
- **EscrowableERC20:** ERC20 token with built-in escrow functionality - enables token-specific escrow capabilities

#### Key Features:

- Immutable core contracts (no proxies) for maximum security and auditability
- Snapshot semantics: Escrow rules, modules, and fees locked at creation, immune to governance changes
- Fee immutability: Protocol fees (yield and appeal bond) snapshotted per-escrow at creation, ensuring fees cannot change during escrow lifetime
- Account abstraction compatible: Works with smart contract wallets and legacy EOAs

#### Resolution Modules (Swappable via Governance)

- **DefaultResolutionModule:** Simple single-resolver system (initial mainnet deployment)
  - Single trusted resolver per escrow
  - Governance-controlled resolver updates
  - Suitable for initial launch and simple disputes
- **DecentralizedResolutionModule:** Advanced multi-resolver system with escalation (governance-activated module swap)
  - Multi-resolver registry with round-robin selection
  - Three-level escalation: Standard → Senior → External resolver
  - Category-based dispute routing
  - Resolver incentive system

**Module Governance:** All modules are immutable. Module upgrades are performed by deploying a new version and swapping via Slow lane (queue + activate, ~9 days). Both queue and activate operations require Timelock execution (ROLE\_TIMELOCK), ensuring all module changes are time-delayed and transparent.

#### Supporting Modules (Optional)

- **AaveYieldGenerationModule:** Generates yield on escrowed funds via Aave integration
  - Optional per-escrow yield generation
  - Protected by exposure caps and pause mechanisms
  - Governance-controlled enable/disable
- **DefaultYieldDistributionModule:** Configurable yield distribution to recipients
  - Percentage-based allocation
  - Multiple recipients support
  - Immutable at escrow creation
- **ResolverIncentiveModule:** Tracks and distributes payments to resolvers (future, with DecentralizedResolutionModule)
  - Resolver activity tracking
  - Automatic payment distribution
  - Incentive alignment for fair resolution

## 2.3 Key Guarantees

The protocol provides the following guarantees:

1. **Fund Safety:** Escrowed funds are held in smart contracts and cannot be accessed without proper authorization
  2. **Rule Immutability:** Once an escrow is created, its rules (modules, timeouts, resolver) cannot be changed by any actor, including governance
  3. **Governance Bounds:** All governance changes are time-delayed and affect only new escrows
  4. **Emergency Controls:** Guardian can pause protocol or reduce risk, but cannot increase risk or unpause
- 

## 3. Protocol Architecture

### 3.1 Core Escrow Contracts

#### BaseEscrow

The abstract base contract provides the foundation for all escrow functionality:

- **Multi-token support:** Escrow any ERC20 token
- **State machine:** Manages escrow lifecycle (Pending → Funded → Disputed → Resolved/Cancelled)
- **Dispute resolution:** Integrates with resolution modules
- **Fee management:** Tracks and collects protocol fees
- **Module integration:** Pluggable resolution, yield, and distribution modules

#### EscrowVault

A multi-token escrow vault that supports escrowing multiple different tokens:

- **Use Case:** When you need to escrow multiple different tokens in a single contract
- **Features:** Per-token fee tracking, batch operations, Aave yield integration
- **Deployment:** Immutable (no proxies)

#### EscrowableERC20

An ERC20 token with built-in escrow functionality:

- **Use Case:** When you want a token that has escrow capabilities built-in
- **Features:** Standard ERC20 functionality + `createEscrow()` function
- **Deployment:** Factory pattern for easy token creation

### 3.2 Resolution Modules

#### DefaultResolutionModule (Initial Deployment)

A simple, single-resolver resolution module:

- **Resolver Assignment:** Single resolver per escrow
- **Governance-Controlled:** Resolver updates require governance approval
- **Use Case:** Simple disputes that don't need escalation

#### DecentralizedResolutionModule (Staged Rollout: DR v1 → v2 → v3)

An advanced decentralized resolution system implementing a **staged rollout approach** following the principle: **"Decentralise decisions first, decentralise incentives second, decentralise capital last."** This minimizes risk by introducing adversarial pressure gradually, only after each phase proves stable.

### Staged Dispute-Resolution Rollout (DR v1 → v3)

The protocol is designed to progressively decentralize dispute resolution. **Only the initial mainnet release configuration is active at launch**; DR modules are **implemented and locally tested**, but **not yet validated on testnet/simulation** and therefore **not activated** until governance phase-gates are met.

#### Initial mainnet release (IEO configuration)

- Uses **DefaultResolutionModule** (single resolver)
- Goal: minimal surface area while core escrow and governance are proven under real usage

#### DR v1 — Decentralize Decisions (multi-resolver escalation)

- **What changes:** multiple resolvers, fair selection, escalation ladder (Standard → Senior → External)
- **What stays centralized:** incentives and capital (no resolver staking/slashing)
- **Why safer:** no resolver capital at risk; performance-based routing and automatic timeout handling reduce reliance on governance intervention

#### DR v2 — Decentralize Incentives (appeal bonds + cost curves)

- **What changes:** users post **appeal bonds** to escalate; costs increase by round (quadratic recommended)
- **Bond outcomes:** refunded if appeal succeeds (decision changes); paid to prior resolver set if appeal fails (decision upheld)
- **What stays centralized:** capital (still no resolver staking/slashing)
- **Why safer:** economic friction suppresses griefing and low-quality escalation without introducing resolver slashing risk

#### DR v3 — Decentralize Capital (staking + objective slashing)

- **What changes:** resolver staking, objective penalties, freezes/caps, fraud slashing path
- **Security model:** oracle-free mixed bond design
  - Composition enforced: ≥80% stablecoin security, ≤20% SEW (post-haircut)
- **Why safer (when activated):** capital at risk aligns incentives, while stablecoin anchoring reduces volatility-driven solvency risk

For launch-safe parameter defaults and rationale (DR v3), see: [docs/dispute-resolution/DR\\_V3\\_LAUNCH\\_SAFE\\_DEFAULTS.md](#).

## 3.3 Yield Generation

### AaveYieldGenerationModule

Optional yield generation on escrowed funds:

- **Integration:** Deposits escrowed funds to Aave for yield
- **Protection:** Exposure caps and pause mechanisms
- **Governance-Controlled:** Can be enabled/disabled via governance

- **Use Case:** Generate yield on funds locked in escrow

### 3.4 Snapshot Semantics

A critical design feature: **module addresses and settings are snapshotted at escrow creation:**

- When an escrow is created, the current module addresses are stored in the escrow record
- These addresses **cannot be changed** after creation
- Governance can swap modules, but this only affects **new escrows**
- Existing escrows continue using their snapshotted modules

This ensures that:

- Governance cannot retroactively change escrow rules
  - Users can trust that their escrow rules will remain unchanged
  - Protocol evolution doesn't break existing escrows
- 

## 4. Key Features

### 4.1 Multi-Token Support

The protocol supports escrowing **any ERC20 token:**

- Standard ERC20 tokens
- Custom tokens with specific features
- Single interface for all tokens
- Per-token fee tracking

### 4.2 Dispute Resolution

Built-in dispute resolution with **staged decentralization** approach. The protocol uses a staged rollout following the principle: "Decentralise decisions first, decentralise incentives second, decentralise capital last." This minimizes risk by introducing adversarial pressure gradually, only after each phase proves stable.

The protocol is designed for staged activation:

- **Initial mainnet release (IEO configuration):** `DefaultResolutionModule` (single resolver) + governance + core escrow
- **DR v1/v2/v3 modules:** implemented and passing local tests, but **not active** at initial mainnet release and **not yet validated** on testnet/simulation at the time of this document. Activation requires a **governance-activated module swap** and affects **new escrows only**.

See Section 3.2 for detailed staging information, phase gates, attack vectors, and risk mitigation strategies.

### 4.3 Optional Yield Generation

Escrowed funds can generate yield via Aave:

- **Optional:** Yield generation can be enabled/disabled per escrow
- **Protected:** Exposure caps and pause mechanisms
- **Transparent:** All yield operations are onchain
- **Distributable:** Yield can be distributed to recipients



- **Important:** Yield is optional per-escrow; if disabled or paused, escrow settlement continues without yield. Yield integration does not change the escrow state machine and is designed to be unwindable under pause controls and caps

## 4.4 Automated Settlements

Time-based automatic settlements:

- **Auto-Release:** Automatic release after timeout (if configured)
- **Auto-Cancel:** Automatic cancellation if not funded within timeout
- **Configurable:** Timeouts can be set per escrow

## 4.5 Attachments & Evidence

Support for dispute evidence:

- **IPFS Integration:** Store evidence on IPFS
- **Hash Verification:** Onchain hash verification
- **Multiple Attachments:** Support for multiple evidence files

## 4.6 Governance-Controlled Evolution

Protocol evolution through onchain governance:

- **Time-Delayed:** All changes require timelock delays
- **Transparent:** All proposals and votes are onchain
- **Bounded:** Changes affect only new escrows
- **Emergency Controls:** Guardian can pause or reduce risk

---

# 5. Governance Model

## 5.1 Governance Structure

The protocol uses a multi-layered governance system:

1. **OpenZeppelin Governor:** Token-based voting for proposals
2. **TimelockController:** Time-delayed execution (48h standard, ~9 days slow lane)
3. **Guardian Multisig:** Emergency controls (down-only)

**Canonical references (exchange/audit):**

- Governance powers, lanes, and delays (authoritative):  
[docs/governance/GOVERNANCE\\_SURFACE\\_MAP.md](#)
- Deployment defaults (token/supply, timelock delay, governor params): [config/governance.config.ts](#)

## 5.2 Governance Lanes

**Emergency Lane (0h delay, Guardian only)**

- Pause protocol
- Disable features (e.g., Aave)
- Lower exposure caps
- **Down-only:** Cannot unpause, enable features, or raise caps

### Standard Lane (48h delay, Timelock)

- Operational parameter changes (non-economic)
- Unpause protocol
- Module allowlists/registrations and routine operational config (bounded)

### Slow Lane (~9 days delay, Timelock)

- Module swaps (all modules use this pattern)
  - Process: Deploy new module → Queue (48h delay via Timelock) → Wait 7 days → Activate (48h delay via Timelock)
  - Total time: ~9 days wall-clock (48h + 7d + 48h)
  - Both queue and activate require Timelock execution (ROLE\_TIMELOCK)
  - All modules are immutable; upgrades are performed by deploying a new version and swapping
- Economic parameter changes (fees, protocol fee recipients, and other high-impact economic settings)
- Fee recipient changes
- Critical parameter changes

## 5.3 Governance Guarantees

1. **No In-Flight Escrow Modification:** Governance cannot change rules for existing escrows
  2. **Snapshot at Creation:** Module addresses locked per escrow
  3. **Time-Delayed Execution:** All non-emergency changes require timelock
  4. **Down-Only Emergency Controls:** Guardian can only reduce risk
- 

## 6. Security

### 6.1 Security Goals

Sew Protocol is designed with the following security goals:

1. **Escrow Correctness:** Funds are correctly tracked and cannot be double-spent or lost due to state machine errors
2. **Immutability of In-Flight Escrows:** Escrow rules (modules, timeouts, resolver) cannot be changed after creation by any actor, including governance
3. **Bounded Governance Changes:** All governance changes are time-delayed and bounded. Changes affect only new escrows
4. **Safe Dispute Resolution:** Dispute resolution is handled by authorized resolvers with proper access control. Resolution logic cannot be manipulated
5. **Safe External Integrations:** External integrations (e.g., Aave) are protected by caps, pause mechanisms, and proper accounting. Failures in external protocols do not result in fund loss
6. **No Per-Escrow Admin Overrides:** No function exists that allows governance to modify individual escrow rules after creation
7. **Guardian Down-Only Powers:** Emergency controls can only reduce risk, never increase it. Guardian cannot unpause, enable features, or raise caps
8. **Reentrancy Protection:** Critical functions use reentrancy guards and follow checks-effects-interactions pattern
9. **Access Control Integrity:** Role-based access control is properly enforced. Deployer roles are revoked after deployment

10. **Time-Delayed Governance:** All non-emergency changes execute through onchain timelock (48 hours for Standard lane, ~9 days for Slow lane)

## 6.2 Security Architecture

### Immutable Core Contracts

- **BaseEscrow, EscrowVault, EscrowableERC20:** Deployed immutably (no proxies)
- **No Upgrade Risk:** Core escrow logic cannot be changed after deployment
- **Auditability:** Immutable contracts are easier to audit and verify

### Immutable Modules

- **All modules are immutable:** Upgrades are performed by deploying a new version and swapping via Slow lane (~9 days)
- **Unified Governance:** All modules use the same governance pattern (queue + activate via Timelock)
- **Timelock-Only Execution:** Both queue and activate operations require Timelock execution (ROLE\_TIMELOCK), ensuring all module changes are time-delayed and transparent
- **Snapshot Semantics:** Module addresses are snapshotted at escrow creation and cannot be changed
- **Process:** Deploy new module → Queue (48h delay via Timelock) → Wait 7 days → Activate (48h delay via Timelock) = ~9 days total

### Governance Security

- **Time-Delayed Execution:** All changes require timelock delays (48h Standard, ~9 days Slow)
- **Bounded Changes:** All parameter changes are bounded onchain
- **Emergency Controls:** Guardian can pause or reduce risk, but cannot increase risk
- **Transparent:** All proposals and votes are onchain and publicly verifiable

### External Integration Security

- **Aave Integration:** Protected by exposure caps and pause mechanisms
- **Cap Enforcement:** Deposits enforce `exposure[token] + amount <= cap[token]`
- **Guardian Controls:** Guardian can disable Aave or lower caps immediately
- **Bounded exposure caps:** External protocol failures are bounded by exposure caps and can be mitigated via pause/withdrawal procedures; residual risk remains within the capped exposure.

## 6.3 Trust Model

### What Users Must Trust

1. **Token contracts:** ERC20 tokens behave as specified (standard ERC20 required)
2. **Chain liveness:** Base mainnet remains operational and accessible
3. **Block timestamps:** `block.timestamp` is reasonably accurate for auto-settlement
4. **External protocols:** If yield enabled, Aave protocol functions correctly
5. **Governance process:** Token holders vote honestly and timelock executes correctly
6. **Resolver behaviour:** Disputes rely on human judgment; the protocol mitigates this via escalation, economic friction (appeal bonds), objective timeouts, performance gating, and (in later phases) staking/slashing

### What Users Do NOT Need to Trust

1. **Team/Developers:** Cannot modify in-flight escrow rules. Cannot unilaterally change modules
2. **Governance:** Cannot change rules of existing escrows. Can only affect new escrows
3. **Guardian:** Cannot steal funds, unpause without timelock, or increase risk. Powers are strictly down-only
4. **Deployer:** All deployer roles are revoked after deployment
5. **Future code changes:** Core contracts are immutable, swappable modules. Module swaps are time-delayed and transparent

## 6.4 Security Measures

### Code Security

- **Comprehensive Testing:** Hardhat + Foundry test suites with high coverage
- **Static Analysis:** Slither analysis configured and run regularly
- **Fuzz Testing:** Foundry fuzz tests for critical paths
- **Formal Verification:** Considered for critical invariants

### Operational Security

- **Emergency Procedures:** Clear runbooks for emergency situations
- **Guardian Multisig:** Hardware wallet-based multisig for emergency controls
- **Incident Response:** Documented procedures for security incidents

### Audit & Verification

- **Security Audits:** Multiple audit phases planned before mainnet
- **Bug Bounties:** Bug bounty program for ongoing security
- **Public Verification:** All contracts verified on Basescan
- **Security Documentation:** Comprehensive security model and threat analysis

## 6.5 Known Risks & Mitigations

### Smart Contract Risks

- **Reentrancy:** Mitigated by ReentrancyGuard and CEI pattern
- **Access Control:** Mitigated by role-based access control and role revocation
- **Integer Overflow:** Mitigated by Solidity 0.8.33 built-in checks
- **Front-Running:** Mitigated by commit-reveal patterns where applicable

### Governance Risks

- **Governance Attacks:** Mitigated by timelock delays and proposal thresholds
- **Malicious Proposals:** Mitigated by voting requirements and timelock delays
- **Guardian Compromise:** Mitigated by multisig and down-only powers

### External Risks

- **Aave Protocol Failure:** Mitigated by caps, pause mechanisms, and withdrawal procedures
- **Token Contract Issues:** Mitigated by standard ERC20 requirement and SafeERC20 usage
- **Chain Issues:** Mitigated by Base L2 reliability and monitoring

### Operational Risks

- **Key Management:** Mitigated by hardware wallets and multisig
- **Human Error:** Mitigated by runbooks, testing, and rehearsals
- **Social Engineering:** Mitigated by security policies and access controls

## 6.6 Security Guarantees

Sew Protocol provides the following security guarantees:

1. **Fund Safety:** Escrowed funds are held in smart contracts and cannot be accessed without proper authorization
  2. **Rule Immutability:** Once an escrow is created, its rules cannot be changed by any actor, including governance
  3. **Governance Bounds:** All governance changes are time-delayed and affect only new escrows
  4. **Emergency Controls:** Guardian can pause protocol or reduce risk, but cannot increase risk or unpause
  5. **No Per-Escrow Overrides:** No governance actor can modify rules for a specific escrow after creation
  6. **Transparent Operations:** All operations are onchain and publicly verifiable
- 

## 7. Use Cases

### 7.1 Everyday Physical Goods Purchases (Primary Use Case)

**Scenario:** Consumer purchases physical goods from seller using Ethereum

1. Buyer creates escrow with seller address and payment amount
2. Buyer funds escrow (using account abstraction wallet or legacy EOA)
3. Seller ships physical goods
4. Buyer receives goods, inspects, and releases escrow
5. If dispute (damaged goods, wrong item, non-delivery): Resolver reviews evidence and makes decision

**Benefits:**

- Buyer protection against fraud and errors
- Seller assurance of payment before shipping
- No centralized intermediary or monitoring
- Works with any ERC20 token
- Account abstraction compatible

**Example:** Purchasing electronics, clothing, collectibles, or any physical product with Ethereum payment protection.

### 7.2 Marketplace Integration

**Scenario:** Online marketplace integrates Sew Protocol for buyer protection

1. Marketplace creates escrow for each transaction
2. Buyer funds escrow at checkout
3. Seller ships goods
4. Buyer confirms receipt and releases escrow
5. If dispute: Marketplace resolver or protocol resolver handles dispute

**Benefits:**

- Built-in buyer protection for marketplaces
- Reduces chargeback risk for sellers
- Transparent, onchain dispute resolution
- No payment processor lock-in

### 7.3 Peer-to-Peer Physical Goods Sales

**Scenario:** Person-to-person sale of physical items (e.g., classified ads, social commerce)

1. Buyer creates escrow with seller address
2. Buyer funds escrow
3. Seller ships or delivers goods
4. Buyer verifies and releases escrow
5. If dispute: Resolver reviews evidence (photos, tracking, messages)

**Benefits:**

- Protection for both parties in P2P transactions
  - No need for trusted intermediary
  - Works with any Ethereum wallet
  - Transparent dispute resolution
- 

## 8. Technical Specifications

### 8.1 Network

- **Primary Network:** Base (Ethereum L2)
- **Chain ID:** 8453 (Base Mainnet)
- **Testnet:** Base Sepolia (Chain ID: 84532)

### 8.2 Smart Contract Infrastructure

- **Solidity Version:** 0.8.33
- **OpenZeppelin Contracts:** v5.4.0

### 8.3 Contract Deployment

- **Core Contracts:** Immutable (no proxies)
- **Modules:** Swappable via governance
- **Deployment Tool:** hardhat-deploy
- **Verification:** Basescan verification supported

### 8.4 Testing

- **Hardhat Tests:** TypeScript-based integration tests
  - **Foundry Tests:** Solidity-based unit and fuzz tests
  - **Coverage:** Comprehensive test coverage
  - **Static Analysis:** Slither configured
- 

## 9. Roadmap

All timeline targets are conditional on audits, drills, and meeting phase-gate metrics under real usage.

## Phase 1: Initial Mainnet Deployment

**Goal:** Launch Sew Protocol on Base mainnet with core escrow functionality

**Deliverables:**

- Core escrow contracts (immutable, no proxies)
- DefaultResolutionModule (single-resolver system)
- Basic governance infrastructure (Governor, Timelock, Guardian)
- Emergency controls (pause, disable Aave, lower caps)
- Aave yield generation integration (optional)
- Comprehensive testing and security audits
- Operational runbooks and emergency procedures

**Status:** Pre-mainnet, awaiting audits and drills

**Timeline:** Q1 2026 (target)

---

## Phase 2: Extensive Simulation and Testing of Decentralized Dispute Resolution Network

**Goal:** Prove DecentralizedResolutionModule in isolation before mainnet integration

**Deliverables:**

- Deploy DecentralizedResolutionModule in separate package
- Extensive testing with simulated disputes
- Resolver network testing and validation
- Performance and gas optimization
- Security audits of decentralized resolution system
- Resolver incentive system testing
- Real-world dispute scenario simulations

**Status:** Module extracted, testing in progress

**Timeline:** Q2-Q3 2026 (target, subject to audits and phase gates)

---

## Phase 3: Mainnet Migration to Decentralized Dispute Resolution Module

**Goal:** Swap DecentralizedResolutionModule into mainnet protocol via governance

**Deliverables:**

- Deploy DecentralizedResolutionModule contract
- Governance proposal to queue module swap (48h delay via Timelock)
- Community vote and timelock execution of queue
- Wait 7 days (slow lane delay enforced onchain)
- Governance proposal to activate module swap (48h delay via Timelock)
- Community vote and timelock execution of activate
- Resolver network onboarding
- Incentive system activation

- Monitoring and optimization
- Status:** Planned for after Phase 2 completion

**Timeline:** Q4 2026 (target, after Phase 2 validation)

**Process:**

1. Deploy new DecentralizedResolutionModule
  2. Queue module swap (48h delay via Timelock) - requires Timelock execution (ROLE\_TIMELOCK)
  3. Wait 7 days (slow lane delay)
  4. Activate module swap (48h delay via Timelock) - requires Timelock execution (ROLE\_TIMELOCK)
  5. Total time: ~9 days wall-clock (48h + 7d + 48h)
- Note:** All modules are immutable. Module upgrades are performed by deploying a new version and swapping via Slow lane (queue + activate via Timelock). Both queue and activate operations require Timelock execution (ROLE\_TIMELOCK), ensuring all module changes are time-delayed and transparent. All existing escrows continue using DefaultResolutionModule (snapshot preserved). Only new escrows use DecentralizedResolutionModule.

## 10. Tokenomics

### 10.1 Fee Structure

**Fees TL;DR (defaults at launch)**

| Fee        | Base                           | Default      | Max (onchain) | Recipient        | Notes                                                                                                |
|------------|--------------------------------|--------------|---------------|------------------|------------------------------------------------------------------------------------------------------|
| Escrow fee | Escrow principal (at creation) | 1% (100 bps) | 2% (200 bps)  | escrowFeeAddress | Collected at creation; <b>immutable per-escrow</b> - affects only new escrows created under that fee |



| Fee                                                                   | Base                                   | Default              | Max<br>(onchain)     | Recipient                     | Notes                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------|----------------------------------------|----------------------|----------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Yield protocol fee<br>( <code>yieldProtocolFeeBps</code> )            | Yield<br>only<br>(never<br>principal)  | 30%<br>(3000<br>bps) | 30%<br>(3000<br>bps) | <code>escrowFeeAddress</code> | Snapshotted<br>per-escrow<br>at creation -<br>fee cannot<br>change<br>during<br>escrow<br>lifetime;<br>deducted<br>before yield<br>distribution;<br>if yield<br>distribution<br>fails, yield<br>can be<br>routed to fee<br>recipient    |
|                                                                       |                                        |                      |                      |                               |                                                                                                                                                                                                                                         |
| Appeal bond protocol fee<br>( <code>appealBondProtocolFeeBps</code> ) | Appeal<br>bond (at<br>posting<br>time) | 0%                   | 30%<br>(3000<br>bps) | <code>escrowFeeAddress</code> | Snapshotted<br>per-escrow<br>at creation -<br>fee cannot<br>change<br>during<br>escrow<br>lifetime;<br>implemented<br>but inactive<br>by default at<br>launch; when<br>enabled,<br>bond<br>accounting<br>uses the<br>post-fee<br>amount |
|                                                                       |                                        |                      |                      |                               |                                                                                                                                                                                                                                         |

Escrow Fees

- **Escrow Fee:** 1% of escrow amount (100 basis points)
- **Fee Recipient:** Protocol treasury (governance-controlled)
- **Collection:** Fees are collected at escrow creation and held in protocol treasury
- **Immutability:** Escrow fee is immutable per-escrow - once set at creation, it cannot change for that escrow, even if governance changes the global escrow fee for new escrows

Yield Distribution

- **Yield Generation:** Optional per-escrow (via Aave integration)
- **Protocol share (default):** 30% of generated yield goes to protocol fee recipient (governance-controlled, bounded)
- **Fee Immutability: Protocol fee on yield (`yieldProtocolFeeBps`) is snapshotted per-escrow at creation** - the fee rate is locked for that escrow and cannot change, even if governance changes the global yield protocol fee for new escrows
- **User share (default):** 70% of generated yield distributed to configured recipients
- **Important:** Yield fees apply to yield only (never escrow principal); if yield is disabled or fails, escrow settlement is unaffected.

## 10.2 Governance Token (SEW)

**Purpose:** Governance token for protocol decision-making

### Governance Functions:

- Proposal creation and voting
- Parameter changes
- Module swaps
- Fee recipient updates
- Emergency response coordination

**Token Distribution:** TBD (to be determined before mainnet launch)

### Voting Mechanics:

- Token-based voting (1 token = 1 vote)
- Quorum requirements
- Proposal thresholds
- Timelock execution

### SEW economic utility beyond governance (when DR v3 is activated)

SEW is designed to have **economic utility** in the decentralized resolver network once DR v3 is **activated** (via governance module swap; affects new escrows only):

- **Mixed-bond requirement (oracle-free):** resolvers/seniors stake a mix of **USDC (stable)** and **SEW**:
  - Effective bond:  $\text{EffectiveBondUSD} = \text{stablecoinBond} + (\text{sewBond} \times 0.5)$
  - Composition enforced: **≥80% stablecoin security / ≤20% SEW** (post-haircut), i.e., “**20% SEW / 80% USDC**” at the margin.
- **Coverage / underwriting mechanics:** seniors can underwrite multiple resolvers subject to on-chain coverage accounting (coverage multiplier + utilization buffer).
- **Deflationary sink on misconduct:** when SEW is **slashed from resolver/senior stake**, it is treated as **burned** (supply-reducing when supported; otherwise effectively burned via transfer to a dead address).  
**Note:** this refers to **slashing of staked SEW** (DR v3 capital), not appeal bonds posted by disputing parties.

**Staking rewards:** any explicit “staking rewards” program (e.g., SEW incentives for staking) is **not assumed by default** and should be treated as **planned / governance-defined** if introduced later (separately specified and disclosed).

## 10.3 Revenue Streams

### Protocol Revenue:

1. **Escrow fees:** 1% of all escrow amounts (immutable per-escrow)
2. **Yield protocol fee:** 30% of generated yield by default (when yield enabled; **snapshotted per-escrow at creation**; governance-controlled, bounded)
3. **Appeal bond protocol fee:** Implemented but **0% by default at launch**; can be enabled later by governance (**snapshotted per-escrow at creation**; bounded; applies at bond posting for new escrows)

**Fee Immutability Guarantee:** All fees (escrow fee, yield protocol fee, and appeal bond protocol fee) are **snapshotted per-escrow at creation time**. Once an escrow is created, its fees are immutable and cannot change, even if governance changes global fee parameters. This ensures predictable economics for users and prevents unexpected fee increases during escrow lifetime.

#### Revenue Use:

- Protocol development and maintenance
- Security audits and bug bounties
- Resolver network incentives (after DecentralizedResolutionModule)
- Governance operations
- Emergency reserves

## 10.4 Incentive Alignment

#### Resolver Incentives (After DecentralizedResolutionModule):

- Appeal-bond incentives: resolvers earn bonds when appeals fail (decision upheld)
- Quality-based payment weighting (by escalation level)
- Activity tracking and rewards (program details, if any, are governance-defined)
- Fair workload distribution via round-robin selection

#### User Benefits:

- Buyer protection for everyday purchases
- Seller payment assurance
- Optional yield generation (70% to users)
- Transparent, onchain dispute resolution

## 11. Risks & Limitations

### 11.1 Known Limitations

1. **Reliance on `block.timestamp`:** Auto-settlement features depend on `block.timestamp`, which can be manipulated by miners/validators within a certain range (typically  $\pm 15$  seconds). This is acceptable for day-level timeouts but should be considered for shorter durations.
2. **Dispute Resolution is a Social Process:** While dispute resolution is executed onchain, the decision-making process (resolver judgment) is inherently social. The protocol provides technical guarantees (access control, time delays, escalation) but cannot eliminate the need for human judgment in disputes.
3. **Governance Can Change Defaults for New Escrows:** By design, governance can change default modules, fees, timeouts, and other parameters. These changes affect only escrows created after the change (snapshot immutability). **Fees are snapshotted per-escrow at creation** - protocol fees (yield and appeal bond) cannot change during an escrow's lifetime, ensuring predictable economics for users.

4. **External Protocol Dependencies:** If yield generation is enabled, the protocol depends on Aave functioning correctly. Aave protocol failures or exploits could affect yield generation, though caps and pause mechanisms limit exposure.
5. **Token Support Limitations:** The protocol assumes standard ERC20 behavior. Non-standard tokens (fee-on-transfer, rebasing) may not be fully supported. See Security Model for details.
6. **Gas Costs:** Complex operations (dispute resolution with multiple payouts, large attachment arrays) may have high gas costs. Users should be aware of gas implications, especially on mainnet.
7. **Account Abstraction Compatibility:** While the protocol works with both account abstraction wallets and legacy EOAs, some advanced features may require smart contract wallet capabilities.

## 11.2 Risk Mitigations

- **Comprehensive Testing:** Extensive test coverage and fuzz testing
- **Security Audits:** Multiple audit phases planned
- **Time-Delayed Governance:** All changes require timelock delays
- **Emergency Controls:** Guardian can pause or reduce risk
- **Immutable Core Contracts:** Core escrow logic cannot be changed

---

## 12. Conclusion

Sew Protocol provides a secure, transparent, and flexible solution to the trust problem in blockchain transactions, specifically enabling safe everyday purchases of physical goods with Ethereum. By combining immutable core contracts with modular, governance-controlled components, the protocol enables trustless peer-to-peer transactions while maintaining the ability to evolve and improve over time.

### Key Differentiators:

1. **Immutability Where It Matters:** Core escrow rules are immutable, but the protocol can evolve
2. **Modular Design:** Pluggable modules enable flexibility without compromising security
3. **Unified Module Governance:** All modules are immutable and use the same Slow lane swap pattern (queue + activate via Timelock, ~9 days). Both queue and activate require Timelock execution, ensuring all module changes are time-delayed and transparent.
4. **Governance-Controlled Evolution:** Transparent, time-delayed protocol upgrades
5. **Comprehensive Security:** Multiple layers of security and emergency controls
6. **User-Centric Design:** Focus on protecting both buyers and sellers in everyday purchases
7. **Account Abstraction Compatible:** Works with smart contract wallets and legacy EOAs
8. **Genuinely Decentralized:** No custodians, no user surveillance, and no platform lock-in — rules are enforced onchain, and upgrades are transparent and time-delayed

**Vision:** Sew Protocol is designed to accelerate consumer adoption of Ethereum for payments by solving the fundamental problem of lost money through errors or fraud. We introduce new primitives that augment account abstraction, enabling safe, trustless transactions for physical goods while maintaining the decentralized, transparent, and user-controlled principles of Web3.

The protocol is designed to become a foundational piece of infrastructure for trustless transactions on Ethereum and Layer 2 networks, specifically enabling safe everyday purchases and improving the user experience for decentralized commerce.

---

## 13. References & Resources

### Documentation

- [Technical Overview](#) - Detailed technical documentation
- [Security Model](#) - Comprehensive security analysis
- [Governance Model](#) - Governance structure and procedures
- [Contracts Summary](#) - Contract overview
- [Upgrade Policy](#) - Upgrade procedures

### Security

- [Security Policy](#) - Security contact and disclosure policy
- [Audit Documentation](#) - Audit status and reports

### Governance

- [Governance Surface Map](#) - Complete function mapping
- [Operational Runbooks](#) - Step-by-step procedures

---

## Appendix A: Glossary

- **Escrow:** Funds held by a third party until conditions are met
- **Snapshot Semantics:** Module addresses locked at escrow creation
- **Resolution Module:** Contract that handles dispute resolution logic
- **Governance Lane:** Different delay periods for different types of changes
- **Timelock:** Time-delayed execution mechanism
- **Guardian:** Emergency control role with down-only powers

---

## Appendix B: Contract Addresses

*To be filled after mainnet deployment*

- **EscrowVault:** TBD
- **EscrowableERC20:** TBD
- **DefaultResolutionModule:** TBD
- **Governor:** TBD
- **TimelockController:** TBD

---

*This whitepaper is a living document and will be updated as the protocol evolves.*